
Coding Questions

1. Print Boundary Nodes of a Binary Tree

Solution:

```
class Solution {
public List<Integer> boundary(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;

    if (!(root.left == null && root.right == null)) {
        ans.add(root.val);
    }

    TreeNode cur = root.left;
    while (cur != null) {
        if (cur.left != null || cur.right != null) {
            ans.add(cur.val);
        }
        cur = (cur.left != null) ? cur.left : cur.right;
    }

    addLeaves(root, ans);

    List<Integer> temp = new ArrayList<>();
    cur = root.right;

    while (cur != null) {
        if (cur.left != null || cur.right != null) {
            temp.add(cur.val);
        }
        cur = (cur.right != null) ? cur.right : cur.left;
    }

    for (int i = temp.size() - 1; i >= 0; i--) {
        ans.add(temp.get(i));
    }

    if (root.left == null && root.right == null) {
        ans.add(root.val);
    }

    return ans;
}

void addLeaves(TreeNode root, List<Integer> ans) {
    if (root == null) return;

    if (root.left == null && root.right == null) {
        ans.add(root.val);
        return;
    }

    addLeaves(root.left, ans);
    addLeaves(root.right, ans);
}
```

```
}
```

2. Left View

Solution :

```
class Solution {
public List<Integer> leftView(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;

    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        int size = q.size();

        for (int i = 0; i < size; i++) {
            TreeNode cur = q.poll();

            if (i == 0) {
                ans.add(cur.val);
            }

            if (cur.left != null) q.add(cur.left);
            if (cur.right != null) q.add(cur.right);
        }
    }

    return ans;
}
}
```

3. Right view

Solution:

```
class Solution {
public List<Integer> rightView(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;

    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        int size = q.size();

        for (int i = 0; i < size; i++) {
            TreeNode cur = q.poll();

            if (i == size - 1) {
                ans.add(cur.val);
            }

            if (cur.left != null) q.add(cur.left);
        }
    }
}
```

```

        if (cur.right != null) q.add(cur.right);
    }
}

return ans;
}
}

```

4. Lowest Common Ancestor

Solution:

```

class Solution {
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) {
        return root;
    }

    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    if (left != null && right != null) {
        return root;
    }

    return left != null ? left : right;
}
}

```

5. Root to Node Path

Solution :

```

class Solution {
public List<Integer> findPath(TreeNode root, int target) {
    List<Integer> path = new ArrayList<>();

    if (find(root, target, path)) {
        return path;
    }

    return new ArrayList<>();
}

boolean find(TreeNode root, int target, List<Integer> path) {
    if (root == null) return false;

    path.add(root.val);

    if (root.val == target) {
        return true;
    }

    if (find(root.left, target, path) ||
        find(root.right, target, path)) {
        return true;
    }
}
}

```

```

        path.remove(path.size() - 1);

        return false;
    }
}

```

6. Construct Tree from Inorder and Preorder

Solution:

```

        class Solution {
        int preIndex = 0;

        public TreeNode buildTree(int[] preorder, int[] inorder) {
            return build(preorder, inorder, 0, inorder.length - 1);
        }

        TreeNode build(int[] preorder, int[] inorder, int left, int right) {
            if (left > right) return null;

            int rootValue = preorder[preIndex++];
            TreeNode root = new TreeNode(rootValue);

            int index = left;

            while (inorder[index] != rootValue) {
                index++;
            }

            root.left = build(preorder, inorder, left, index - 1);
            root.right = build(preorder, inorder, index + 1, right);

            return root;
        }
    }
}

```